

딥러닝 파이토치 교과서 5장 (~5.2)

5장 합성곱 신경망 I

5.1 합성곱 신경망

딥러닝의 역전파는 순전파 과정에 따라 계산된 오차 정보가 신경망의 모든 노드(출력층 → 은닉층 → 입력층)로 전송된다. 이러한 계산 과정은 복잡하고 많은 자원(CPU 혹은 GPU, 메모리)을 요구하며, 계산하는 데도 오래 걸린다.

이 문제를 해결하고자 하는 것이 합성곱 신경망이다. 합성곱 신경망은 이미지 전체를 한 번에 계산하는 것이 아닌 이미지의 국소적 부분을 계산함으로써 시간과 자원을 절약하여 이미지의 세밀한 부분까지 분석할 수 있는 신경망이다.

5.1.1 합성곱층의 필요성

- 합성곱 신경망은 이미지나 영상을 처리하는 데 유용하다.
- 이미지 분석은 3×3 배열을 펼쳐서(flattening) 각 픽셀에 가중치를 곱하여 은닉층으로 전달하게 된다.
- 하지만 이미지를 펼쳐서 분석하면 데이터의 공간적 구조를 무시하게 되는데, 이것을 방지하려고 도입된 것이 합성곱층이다.

5.1.2 합성곱 신경망 구조

합성곱 신경망(CNN 또는 ConvNet)은 음성 인식이나 이미지/영상 인식에서 주로 사용되는 신경망이다. 다차원 배열 데이터를 처리하도록 구성되어 컬러 이미지 같은 다차원 배열 처리에 특화되어 있으며, 다음과 같이 계층 다섯 개로 구성된다.

1. 입력층
2. 합성곱층
3. 풀링층
4. 완전연결층
5. 출력층

합성곱 신경망은 합성곱층과 풀링층을 거치면서 입력 이미지의 주요 특성 벡터를 추출한다. 그 후 추출된 주요 특성 벡터들은 완전연결층을 거치면서 1차원 벡터로 변환되며, 마지막으로 출력층에서 활성화 함수인 소프트맥스 함수를 사용하여 최종 결과가 출력된다.

입력층

- 입력 이미지 데이터가 최초로 거치게 되는 계층이다.
- 이미지는 단순 1차원의 데이터가 아닌 높이(height), 너비(width), 채널(channel)의 값을 갖는 3차원 데이터다.
- 채널은 이미지가 그레이스케일이면 1 값을 가지며, 컬러(RGB)이면 3 값을 갖는다.
- 예를 들어 높이 4, 너비 4, 채널은 RGB를 갖고 있다면 이미지 형태(shape)는 (4, 4, 3)으로 표현할 수 있다.

합성곱층

합성곱층은 입력 데이터에서 특성을 추출하는 역할을 수행한다.

- 입력 이미지가 들어왔을 때 이미지에 대한 특성을 감지하기 위해 커널(kernel)이나 필터를 사용한다.
- 커널/필터는 이미지의 모든 영역을 훑으면서 특성을 추출하게 되는데, 이렇게 추출된 결과물이 특성 맵(feature map)이다.
- 커널은 3×3 , 5×5 크기로 적용되는 것이 일반적이며, 스트라이드(stride)라는 지정된 간격에 따라 순차적으로 이동한다.
- 스트라이드란 필터를 적용하는 위치의 간격을 의미한다.
- 입력 이미지와 필터를 포개 놓고 대응되는 숫자끼리 곱한 후 모두 더한다.
- 커널은 스트라이드 간격만큼 순회하면서 모든 입력 값과의 합성곱 연산으로 새로운 특성 맵을 만들게 되며, 커널과 스트라이드의 상호 작용으로 원본 크기가 더 작은 크기의 특성 맵으로 줄어든다.

컬러 이미지의 합성곱이 그레이스케일 이미지와 구분되는 특징은 다음과 같다. 그 외 스트라이드 및 연산하는 방법은 동일하다.

1. 필터 채널이 3이다.
2. RGB 각각에 서로 다른 가중치로 합성곱을 적용한 후 결과를 더해 준다.

이때 필터 채널이 3이라고 해서 필터 개수도 세 개라고 오해하기 쉬운데, 실제로는 필터 개수가 한 개라는 점에 주의해야 한다.

필터가 두 개 이상이면 필터 각각은 특성 추출 결과의 채널이 된다. 각 계산은 앞서 진행했던 방법과 동일하다.

합성곱층을 요약하면 다음과 같다.

- 입력 데이터: $W1 \times H1 \times D1$ ($W1$: 가로, $H1$: 세로, $D1$: 채널 또는 깊이)
- 하이퍼파라미터
 - 필터 개수: K
 - 필터 크기: F
 - 스트라이드: S
 - 패딩: P
- 출력 데이터
 - $W2 = (W1 - F + 2P) / S + 1$
 - $H2 = (H1 - F + 2P) / S + 1$
 - $D2 = K$

풀링층

풀링층은 합성곱층과 유사하게 특성 맵의 차원을 다운 샘플링하여 연산량을 감소시키고, 주요한 특성 벡터를 추출하여 학습을 효과적으로 할 수 있게 한다.

- 다운 샘플링(sub-sampling)은 이미지를 축소하는 것이다.

풀링 연산에는 두 가지가 사용된다.

- 최대 풀링(max pooling): 대상 영역에서 최댓값을 추출
- 평균 풀링(average pooling): 대상 영역에서 평균을 반환

하지만 대부분의 합성곱 신경망에서는 최대 풀링이 사용되는데, 평균 풀링은 각 커널 값을 평균화시켜 중요한 가중치를 갖는 값의 특성이 희미해질 수 있기 때문이다.

최대 풀링과 평균 풀링을 요약하면 다음과 같다. 두 풀링의 계산 과정은 다르지만 사용하는 파라미터는 동일하다.

- 입력 데이터: $W1 \times H1 \times D1$
- 하이퍼파라미터
 - 필터 크기: F
 - 스트라이드: S
- 출력 데이터
 - $W2 = (W1 - F) / S + 1$
 - $H2 = (H1 - F) / S + 1$
 - $D2 = D1$

완전연결층

- 합성곱층과 풀링층을 거치면서 차원이 축소된 특성 맵은 최종적으로 완전연결층(fully connected layer)으로 전달된다.
- 이 과정에서 이미지는 3차원 벡터에서 1차원 벡터로 펼쳐지게(flatten) 된다.

출력층

- 출력층에서는 소프트맥스 활성화 함수가 사용되는데, 입력받은 값을 0~1 사이의 값으로 출력한다.
- 마지막 출력층의 소프트맥스 함수를 사용하여 이미지가 각 레이블에 속할 확률 값이 출력되며, 이때 가장 높은 확률 값을 갖는 레이블이 최종 값으로 선정된다.

5.1.3 1D, 2D, 3D 합성곱

합성곱은 이동하는 방향의 수와 출력 형태에 따라 1D, 2D, 3D로 분류할 수 있다.

1. 1D 합성곱

- 필터가 시간을 축으로 좌우로만 이동할 수 있는 합성곱이다.
- 입력(W)과 필터(k)에 대한 출력은 W 가 된다.
- 출력 형태는 1D의 배열이 되며, 그래프 곡선을 완화할 때 많이 사용된다.

2. 2D 합성곱

- 필터가 방향 두 개로 움직이는 형태다.
- 입력(W, H)과 필터(k, k)에 대한 출력은 (W, H)가 되며, 출력 형태는 2D 행렬이 된다.

3. 3D 합성곱

- 필터가 움직이는 방향이 세 개 있다.

- 입력(W, H, L)에 대해 필터(k, k, d)를 적용하면 출력으로 (W, H, L)을 갖는 형태다.
- 출력은 3D 형태이며, 이때 $d < L$ 을 유지하는 것이 중요하다.

4. 3D 입력을 갖는 2D 합성곱

- 입력이 3D 형태임에도 출력 형태가 3D가 아닌 2D 행렬을 취하는 것이다.
- 필터에 대한 길이(L)가 입력 채널의 길이(L)와 같아야 하기 때문에 이와 같은 합성곱 형태가 만들어진다.
- 입력(W, H, L)에 필터(k, k, L)를 적용하면 출력은 (W, H)가 된다. 이때 필터는 두 방향으로 움직이며 출력 형태는 2D 행렬이 된다.
- 대표적 사례로는 LeNet-5와 VGG가 있다.

5. 1×1 합성곱

- 3D 형태로 입력된다.
- 입력(W, H, L)에 필터(1, 1, L)를 적용하면 출력은 (W, H)가 된다.
- 채널 수를 조정해서 연산량이 감소되는 효과가 있으며, 대표적 사례로는 GoogLeNet이 있다.

5.2 합성곱 신경망 맛보기

데이터 불러오기

- torchvision.datasets는 torch.utils.data.Dataset의 하위 클래스로 다양한 데이터셋(CIFAR, COCO, MNIST, ImageNet 등)을 포함한다.
- torch.utils.data.DataLoader()를 사용하여 원하는 크기의 배치 단위로 데이터를 불러오거나, 순서가 무작위로 섞이도록(shuffle) 할 수 있다.

GPU 사용

- 파이토치는 기본적으로 GPU 사용을 권장하지만, GPU가 장착되지 않은 환경에서도 파이토치를 정상적으로 실행하고 사용할 수 있다.
- 사용하는 PC에서 다수의 GPU를 사용한다면 nn.DataParallel을 이용한다.
- nn.DataParallel을 사용할 경우 배치 크기가 알아서 각 GPU로 분배되는 방식으로 작동한다. 따라서 GPU 수 만큼 배치 크기도 늘려 주어야 한다.

객체와 클래스

- 파이토치의 근간은 C++이다. 따라서 C++에서 사용하는 객체 지향 프로그램의 특징들을 파이토치에서도 사용하게 되는데, 대표적인 것이 객체라는 개념이다.
- 객체 지향 프로그래밍은 프로그래밍에서 필요한 데이터를 추상화하여 속성이나 행동, 동작, 특징 등을 객체로 만들고, 그 객체들이 서로 유기적으로 동작하도록 하는 프로그래밍 방법이다.
- 클래스라는 봉어빵 틀에서 여러 개의 객체라는 봉어빵을 찍어 내는 것과 같다. 즉, 재사용성의 이유로 객체 지향 프로그래밍을 많이 사용한다.
- 객체란 메모리를 할당받아 프로그램에서 사용되는 모든 데이터를 의미하기 때문에 변수, 함수 등은 모두 객체라고 할 수 있다.

- 함수란 하나의 특정 작업을 수행하기 위해 독립적으로 설계된 프로그램 코드다. 함수를 포함한 프로그램 코드의 일부를 재사용하기 위해서는 해당 함수뿐만 아니라 데이터가 저장되는 변수까지도 한꺼번에 관리해야 한다.
- 이처럼 함수뿐만 아니라 관련된 변수까지도 한꺼번에 묶어서 관리하고 재사용할 수 있게 해 주는 것이 클래스다.

심층 신경망 모델 구성

1. 클래스 형태의 모델

- 클래스 형태의 모델은 항상 `torch.nn.Module`을 상속받는다.
- `__init__()`은 객체가 갖는 속성 값을 초기화하는 역할을 하며, 객체가 생성될 때 자동으로 호출된다.
- `super(FashionDNN, self).__init__()`은 FashionDNN이라는 부모 클래스를 상속받겠다는 의미로 이해하면 된다.

2. nn.Linear

- nn은 딥러닝 모델 구성에 필요한 모듈이 모여 있는 패키지이며, Linear는 단순 선형 회귀 모델을 만들 때 사용한다.
- `in_features`: 입력의 크기
- `out_features`: 출력의 크기
- 실제로 데이터 연산이 진행되는 `forward()` 부분에는 첫 번째 파라미터 값만 넘겨주게 되며, 두 번째 파라미터에서 정의된 크기가 `forward()` 연산의 결과가 된다.

3. torch.nn.Dropout(p)

- p만큼의 비율로 텐서의 값이 0이 되고, 0이 되지 않는 값들은 기존 값에 $(1/(1-p))$ 만큼 곱해져 커진다.

4. forward()

- 모델이 학습 데이터를 입력받아서 순전파 학습을 진행시키며, 반드시 `forward`라는 이름의 함수여야 한다.
- 객체를 데이터와 함께 호출하면 자동으로 실행된다.
- 순전파 연산이란 $H(x)$ 식에 입력 x 로부터 예측된 y 를 얻는 것이다.

5. view

- 넘파이의 `reshape`과 같은 역할로 텐서의 크기(shape)를 변경해 주는 역할을 한다.
- 첫 번째 차원이 -1이면 사용자가 잘 모르겠으니 파이토치에 맡기겠다는 의미다.

6. 활성화 함수 지정 방법

- `F.relu()`: `forward()` 함수에서 정의
- `nn.ReLU()`: `__init__()` 함수에서 정의

nn.xx와 nn.functional.xx 비교

활성화 함수 사용에 한정하여 이 둘 간의 차이는 간단히 사용하는 위치라고 할 수 있다. 하지만 근본적으로는 사용 방법에 차이가 있다.

1. 형태

- nn.xx: nn.Conv2d처럼 클래스이며, nn.Module 클래스를 상속받아 사용

- `nn.functional.xx`: `nn.functional.conv2d`처럼 함수이며, `def function(input)`으로 정의된 순수한 함수

2. 호출 방법

- `nn.xx`: 먼저 하이퍼파라미터를 전달한 후 함수 호출을 통해 데이터 전달
- `nn.functional.xx`: 함수를 호출할 때 하이퍼파라미터, 데이터 전달

3. 위치

- `nn.xx`: `nn.Sequential` 내에 위치
- `nn.functional.xx`: `nn.Sequential`에 위치할 수 없음

4. 파라미터

- `nn.xx`: 파라미터를 새로 정의할 필요 없음
- `nn.functional.xx`: 가중치를 수동으로 전달해야 할 때마다 자체 가중치를 정의

모델 학습

- 모델이 데이터를 처리하기 위해서는 모델과 데이터가 동일한 장치(CPU 또는 GPU)에 있어야 한다. CPU에서 처리된 데이터를 GPU 모델에 적용하거나 그 반대의 경우 런타임 오류가 발생한다.
- Autograd는 자동 미분을 수행하는 파이토치의 핵심 패키지로, 자동 미분에 대한 값을 저장하기 위해 테이프(tape)를 사용한다.
 - 순전파 단계에서 테이프는 수행하는 모든 연산을 저장한다.
 - 역전파 단계에서 저장된 값들을 꺼내서 사용한다.
 - Autograd는 Variable을 사용해서 역전파를 위한 미분 값을 자동으로 계산해 준다.

분류 문제의 정확도

- 분류 문제에 대한 정확도는 전체 예측에 대한 정확한 예측의 비율로 표현할 수 있다.
 - $\text{classification accuracy} = \frac{\text{correct predictions}}{\text{total predictions}} \times 100$
- 값을 반전시켜 오분류율 또는 오류율로 표현할 수 있다.
 - $\text{error rate} = (1 - (\text{correct predictions} / \text{total predictions})) \times 100$

분류 문제에서 클래스가 세 개 이상일 때는 다음과 같은 사항에 주의해야 한다.

1. 정확도가 80% 이상이라도, 80%라는 값이 모든 클래스가 동등하게 고려된 것인지, 특정 클래스의 분류가 높았던 것인지에 대해 알 수 없다.
2. 100개의 데이터 중 90개가 하나의 클래스에 속할 경우 90%의 정확도는 높다고 할 수 없다. 모든 데이터를 특정 클래스에 속한다고 예측해도 90%의 예측 결과가 나오기 때문에 데이터 특성에 따라 정확도를 잘 관측해야 한다.

합성곱 네트워크 구성

1. nn.Sequential

- `__init__()`에서 사용할 네트워크 모델들을 정의해 줄 뿐만 아니라, `forward()` 함수에서 구현될 순전파를 계층 형태로 좀 더 가독성이 뛰어난 코드로 작성할 수 있다.

- 계층을 차례로 쌓을 수 있도록 $Wx + b$ 와 같은 수식과 활성화 함수를 연결해 주는 역할을 한다.
- 데이터가 각 계층을 순차적으로 지나갈 때 사용하면 좋다.
- 여러 개의 계층을 하나의 컨테이너에 구현하는 방법이라고 생각하면 된다.

2. 합성곱층(nn.Conv2d)

- 합성곱 연산을 통해서 이미지의 특징을 추출한다.
- 합성곱이란 커널(또는 필터)이라는 $n \times m$ 크기의 행렬이 높이 $\times$ 너비 크기의 이미지를 처음부터 끝까지 훑으면서 각 원소 값끼리 곱한 후 모두 더한 값을 출력하는 것이다.
- in_channels: 입력 채널의 수. 흑백 이미지는 1, RGB 값을 가진 이미지는 3을 가진 경우가 많다.
- out_channels: 출력 채널의 수
- kernel_size: 커널 크기이며 논문에 따라 필터라고도 한다. 커널은 이미지 특징을 찾아내기 위한 공용 파라미터이며, CNN에서 학습 대상은 필터 파라미터가 된다. kernel_size=3이면 (3, 3) 정사각형을 의미하며, 직사각형을 사용하고 싶다면 (3, 5)처럼 지정한다.
- padding: 출력 크기를 조정하기 위해 입력 데이터 주위에 0을 채운다. 패딩 값이 클수록 출력 크기도 커진다.

3. 채널

- 흑백 이미지는 $w \times h$ 형태의 행렬로 표현된다.
- 컬러 이미지는 일반적으로 Red, Green, Blue라는 세 개의 채널을 가지며, $w \times h \times c$ 형태의 행렬로 표현된다(c는 채널 수).
- 3차원으로 생각하면 채널은 결국 깊이(depth)를 의미한다.

4. BatchNorm2d

- 학습 과정에서 각 배치 단위별로 데이터가 다양한 분포를 가지더라도 평균과 분산을 이용하여 정규화하는 것을 의미한다.
- 정규화를 통해 분포를 가우시안 형태로 만들며, 평균은 0, 표준편차는 1로 데이터의 분포가 조정된다.

5. MaxPool2d

- 이미지 크기를 축소시키는 용도로 사용한다.
- 풀링 계층은 합성곱층의 출력 데이터를 입력으로 받아서 출력 데이터(activation map)의 크기를 줄이거나 특정 데이터를 강조하는 용도로 사용된다.
- 처리 방법으로는 최대 풀링, 평균 풀링, 최소 풀링이 있다.
- kernel_size: $m \times n$ 행렬로 구성된 가중치
- stride: 입력 데이터에 커널(필터)을 적용할 때 이동할 간격. 스트라이드 값이 커지면 출력 크기는 작아진다.

6. 완전연결층(nn.Linear)

- 클래스를 분류하기 위해서는 이미지 형태의 데이터를 배열 형태로 변환하여 작업해야 한다.
- Conv2d에서 사용하는 하이퍼파라미터 값(패딩, 스트라이드)에 따라 출력 크기가 달라지며, 이렇게 줄어든 출력 크기는 최종적으로 분류를 담당하는 완전연결층으로 전달된다.
- in_features: 입력 데이터의 크기. Conv2d, MaxPool2d의 출력 결과를 완전연결층으로 보내기 위해서는 1차원으로 변경해 주어야 한다.

- out_features: 출력 데이터의 크기

7. view(out.size(0), -1)

- 합성곱층에서 완전연결층으로 변경되기 때문에 데이터의 형태를 1차원으로 바꾸어 준다.
- '-1'은 행의 수는 정확히 알고 있지만 열의 수를 알지 못할 때 사용한다.

출력 크기 구하는 공식

1. Conv2d 계층

- 출력 크기 = $(W - F + 2P) / S + 1$
- W: 입력 데이터의 크기
- F: 커널 크기
- P: 패딩 크기
- S: 스트라이드
- stride가 명시되어 있지 않다면 기본값은 (1, 1)이다.

2. MaxPool2d 계층

- 출력 크기 = IF / F
- IF: 입력 필터의 크기(바로 앞의 Conv2d의 출력 크기이기도 하다)
- F: 커널 크기